

Responsbales : Jose FULLANA
Alice MARCOTTE

SORBONNE UNIVERSITÉ

PROJET - MOTIFS
BACTERIES

Hugo LAM
Zainab ISHTIAQ

2025-2026

Problématiques du projet

- **Comment un modèle simple peut-il générer des structures complexes (patterns) ?**
- **Comment la vitesse de diffusion et la dynamique de croissance influencent-elles la morphologie des structures ?**
- **Quelles conditions ou choix des paramètres physiques déterminent l'apparition, l'absence ou la forme finale des motifs ?**
- **Comment une population bactérienne peut-elle s'organiser spatialement (taches, bandes, labyrinthes) sans signal externe ?**

Sommaire

Modèle Grey-Scott

- Equation mathématiques
- Adimensionnement
- CI et CL
- Discretisation
- Simulation Numérique

Modèle Kawazaki-Murray

- Equations mathématiques
- Non linéarité
- Simulation numérique

Conclusion

Modèle mathématiques de Grey - Scott

On partira d'abord de l'équation générale de réaction-diffusion de Murray donné dans le sujet :

$$\begin{cases} \dfrac{\partial u}{\partial t} = D_u \nabla^2 u + f(u,v), \\ \dfrac{\partial v}{\partial t} = D_v \nabla^2 v + g(u,v). \end{cases}$$

le modèle de Gray-Scott est de la forme :

$$\begin{cases} \dfrac{\partial u}{\partial t} = D_u \nabla^2 u - u\,v^2 + F(1-u) \\ \dfrac{\partial v}{\partial t} = D_v \nabla^2 v + u\,v^2 - (F+k)\,v \end{cases}$$

k = Perte des bactéries ("Kill rate", adimensionnée)

F = Apport en nutriment ("Feed rate", adimensionnée)

Adimensionnement

U_0 = Concentration de réference des nutriments/substrats

V_0 = Densité de référence des bactéries

On définit donc les variables adimensionnées u et v de la manière suivante :

$$u = \frac{U}{U_0} \text{ et } v = \frac{V}{V_0}$$

Conditions Limites

Pour ce problème de réaction-diffusion, on utilisera des conditions aux limites de type Neumann. En effet, on considérera que le bord agit comme une frontière imperméable où les nutriments ne passe pas à travers. La diffusion à travers est nulle.

$$\begin{cases} \frac{\partial u}{\partial x}(0,y,t) = 0 & \frac{\partial u}{\partial x}(N,y,t) = 0 \\ \frac{\partial u}{\partial y}(x,0,t) = 0 & \frac{\partial u}{\partial y}(x,N,t) = 0 \end{cases}$$

Et

$$\begin{cases} \frac{\partial v}{\partial x}(0,y,t) = 0 & \frac{\partial v}{\partial x}(N,y,t) = 0 \\ \frac{\partial v}{\partial y}(x,0,t) = 0 & \frac{\partial v}{\partial y}(x,N,t) = 0 \end{cases}$$

```
Zp = np.pad(Z, pad_width=1, mode='edge')
```

Conditions Initiales

Ainsi, on n'utilisera pas de condition aux limites de type Dirchlet nulle au bord du domaine comme par exemple :

$v(x,0) = 0 \, \forall x \in \in [0,N]$

$u(N,y) = 0 \, \forall y \in \in [0,N]$

```
#Initialisation de u et v
u = np.ones((N, N))
v = np.zeros((N, N))

#CI carré
v[a:b, a:b] = 0.25
u[a:b, a:b] = 0.50
```

Discrétisation Spatiale

Comme pour le TP5, on va utiliser un schema aux différences finis à 5 points (Laplacien à 5 points) :

$$\nabla^2 u(x_i, y_j, t^n) \approx \frac{u^n_{i+1,j} + u^n_{i-1,j} + u^n_{i,j+1} + u^n_{i,j-1} - 4u^n_{i,j}}{h^2}$$

$$\nabla^2 v(x_i, y_j, t^n) \approx \frac{v_{i+1,j} + v_{i-1,j} + v_{i,j+1} + v_{i,j-1} - 4v^n_{i,j}}{h^2}$$

```
#Schéma aux différence finis à 5 points
L = (-4 * Zp[1:-1, 1:-1]
       + Zp[2:, 1:-1]    #i+1, j
       + Zp[:-2, 1:-1]   #i-1, j
       + Zp[1:-1, 2:]    #i, j+1
       + Zp[1:-1, :-2])  #i, j-1

return L / h**2
```

Discrétisation Temporelle

Pour la discrétisation en temps, on utilisera un schéma d'Euler explicite (on remplaçera $f(u, v)$ et $g(u, v)$ par leurs expressions) :

$$u_{i,j}^{n+1} = u_{i,j}^{n} + \Delta t \left(D_u \nabla^2 u_{i,j}^{n} + f(u_{i,j}^{n}, v_{i,j}^{n}) \right)$$

et

$$v_{i,j}^{n+1} = v_{i,j}^{n} + \Delta t \left(D_v \nabla^2 v_{i,j}^{n} + g(u_{i,j}^{n}, v_{i,j}^{n}) \right)$$

```
#Méthode Euler explicite
u_new = u + dt * (Du * Lu + FU)
v_new = v + dt * (Dv * Lv + GV)

#Bornage entre 0 et 1
u_new = np.clip(u_new, 0.0, 1.0)
v_new = np.clip(v_new, 0.0, 1.0)
```

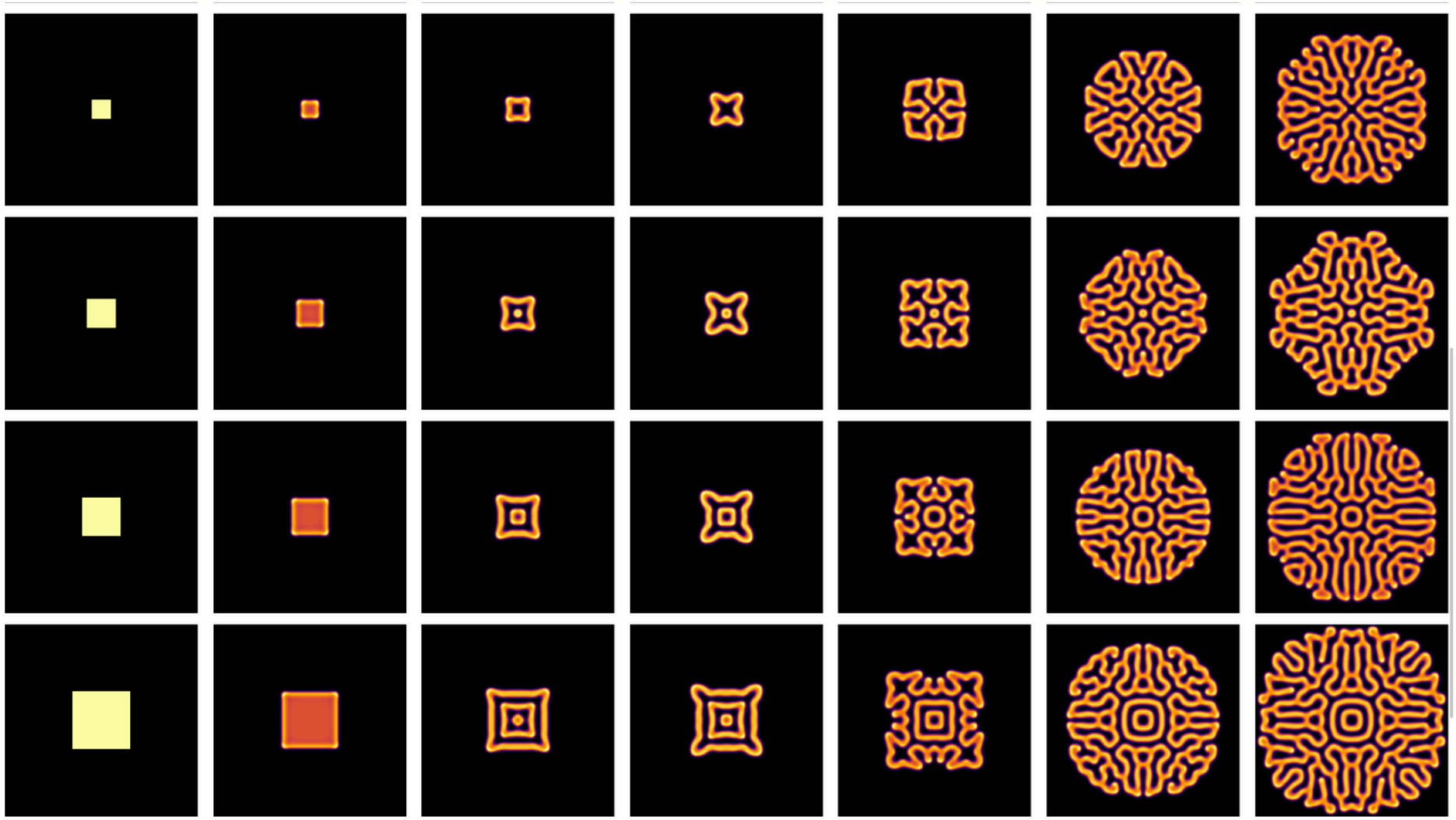
Différentes Tailles Initiales de la Bactérie

```
#Liste des différentes graines (CI) qu'on testera
seed_list = [
    (99, 100),   #1x1 (point au centre), Test 1
    (97, 102),   #5x5 pixels, Test 2
    (95, 105),   #11x11 pixels, Test 3
    (90, 110),   #20x20 pixels, Test 4
    (85, 115),   #30x30 pixels, Test 5
    (80, 120),   #40x40 pixels, Test 6
    (70, 130),   #60x60 pixels, Test 7
    (60, 140)    #70x70 pixels, Test 8
]
all_frames = []  #Liste qui conserve toutes les frames pour toutes les CI
```

```
frames = []    #Liste qui conserve les différentes frames "enregistrées" pour une CI
for t in range(steps + 1):
    if t in checkpoints:
        frames.append(v.copy())
    u, v = step(u, v)

all_frames.append(frames)
```

Motifs Obtenus



Différentes Valeurs de F et K - Stable

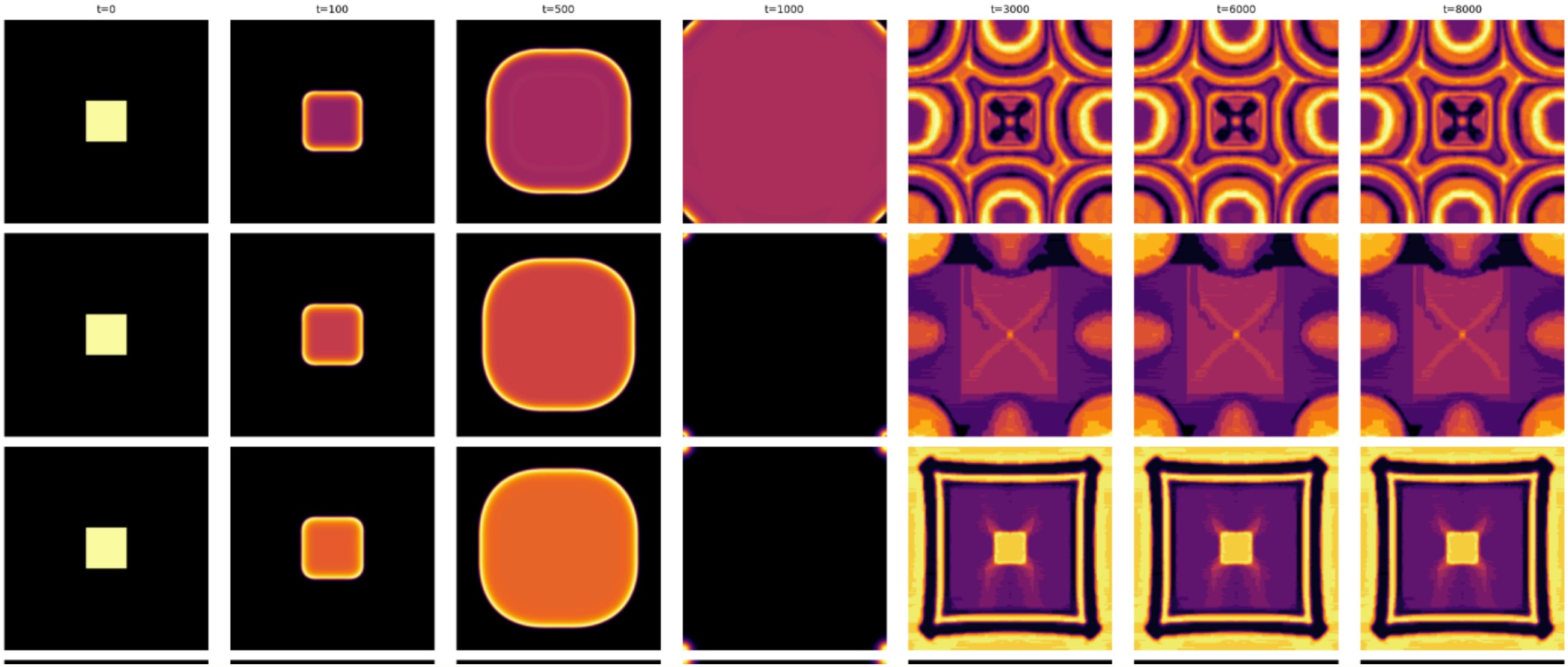
```
Fk_value = [
    (0.015, 0.030), #Test 1
    (0.020, 0.025), #Test 2
    (0.025, 0.020), #Test 3
    (0.030, 0.015)  #Test 4

]

#Fonction qui définit la CI (40x40 pixels)
def initialisation_CI():
    u = np.ones((N, N))
    v = np.zeros((N, N))
    a, b = 80, 120  #Carré 40×40 pixels
    u[a:b, a:b] = 0.50
    v[a:b, a:b] = 0.25
    return u, v
```

```
for (F, k) in Fk_value:
    print(f"Simulation pour F={F}, k={k}")
```

Motifs Obtenus

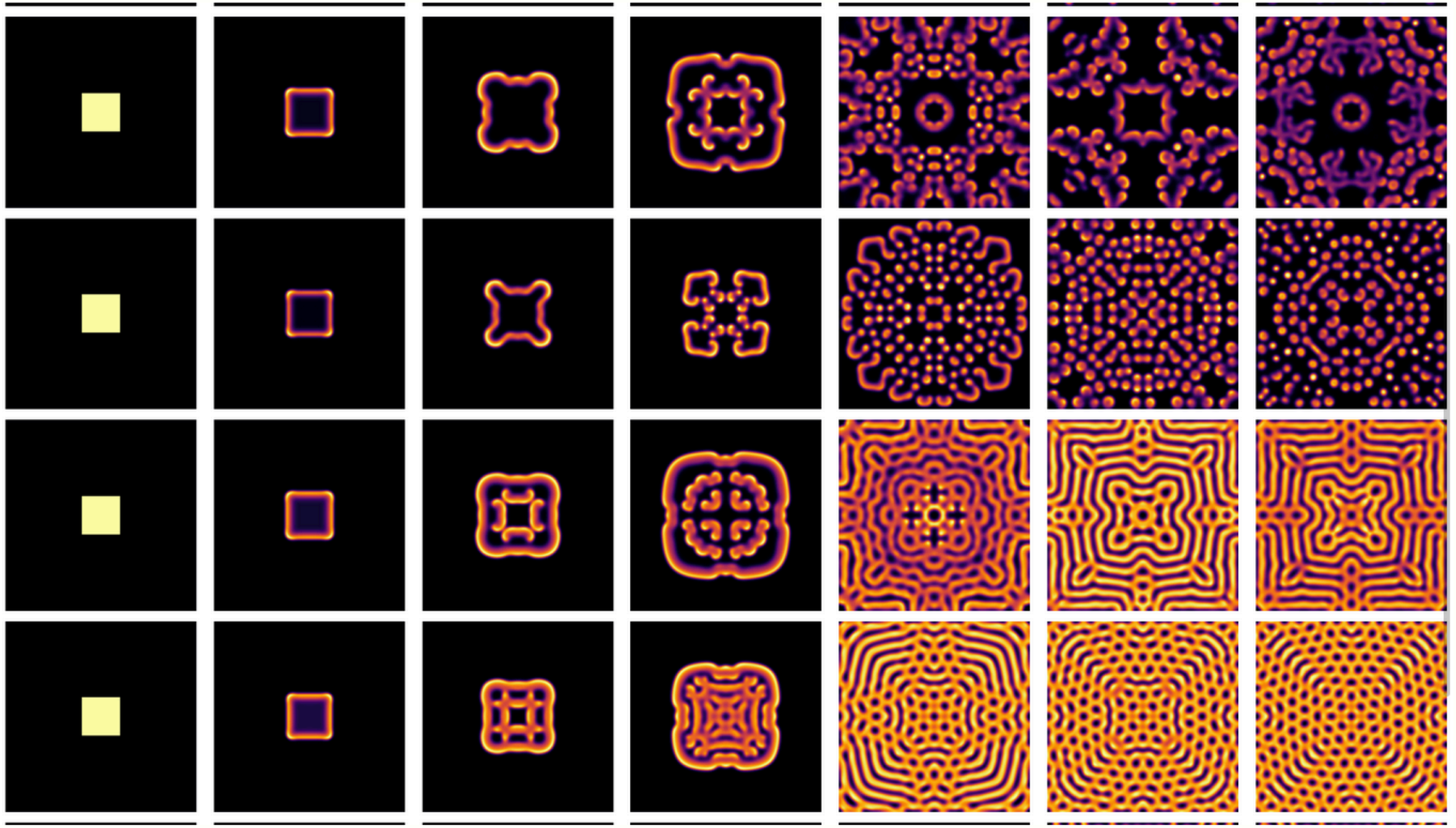


Différentes Valeurs de F et K - Instable

```
#Liste des différentes valeurs des paramétres F (Apport en nutriment) et k (Perte des bactéries) - Instable
Fk_value = [
    (0.014, 0.054), #Test 1 : Pearson (Spotty regime)
    (0.018, 0.051), #Test 2
    (0.020, 0.055), #Test 3
    (0.022, 0.051), #Test 4 : Pearson (Labyrinthe)
    (0.030, 0.055), #Test 5 : Pearson (Labyrinthe)
    (0.040, 0.060), #Test 6 : Pearson (Labyrinthes)
]
```

```
for (F, k) in Fk_value:
    print(f"Simulation pour F={F}, k={k}")
```

Motifs Obtenus



Différentes Valeurs de Du et Dv

```
#Paramètres F et k fixé (choisis aléatoirement parmis les Tests précédent)
F = 0.022
k = 0.051

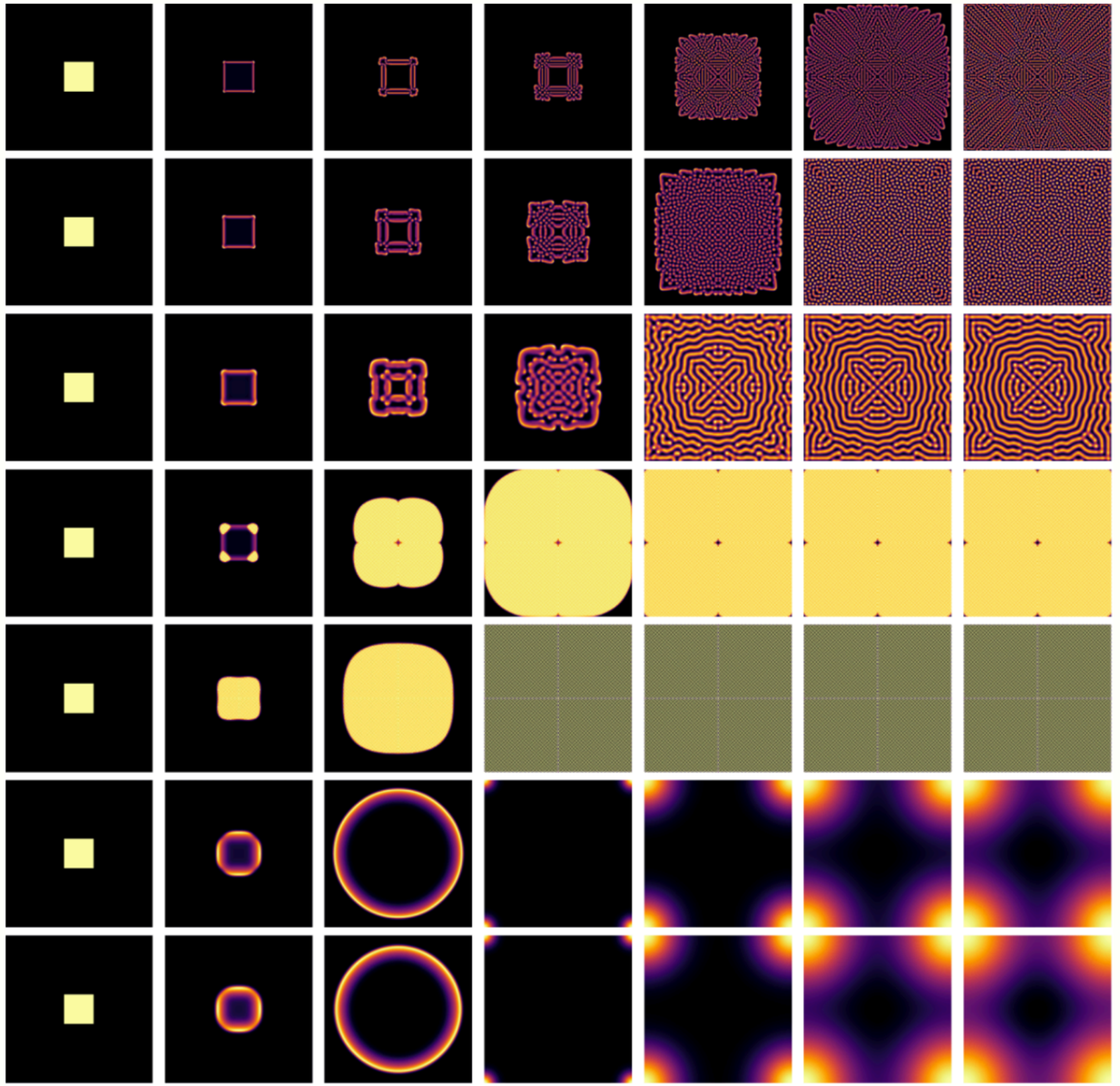
#Liste des différentes valeurs des paramètres Du et Dv choisis (on a choisi des
DuDv_value = [
    (0.08, 0.01),   #Test 1
    (0.16, 0.02),   #Test 2
    (0.18, 0.05),   #Test 3
    (0.24, 0.06),   #Test 4
    (0.32, 0.10),   #Test 5
    (0.05, 0.15),   #Test 6 : On teste Dv >> Du
    (0.10, 0.20),   #Test 7 : On teste Dv >> Du
]
```

```
all_frames = []  #Liste qui conserve toutes les frames pour chaque (Du, Dv) choisis

for (Du_val, Dv_val) in DuDv_value:
    print(f"Simulation pour Du={Du_val}, Dv={Dv_val}")

    #On modifie les valeurs des variables Du et Dv
    Du = Du_val
    Dv = Dv_val
```

Motifs Obtenus



Modèle mathématiques de Kawazaki Murray

Voici les deux équations adimensionnées que nous avons utilisées pour la simulation.

Concentration de nutriment

$$\frac{\partial n}{\partial t} = \nabla^2 n - nb \quad (1)$$

$$\frac{\partial b}{\partial t} = \nabla \cdot (\sigma(x)\, n\, b\, \nabla b) + nb \quad (2)$$

Densité bactérienne

Motilité

Diffusion Bactérienne Non Linéaire

On postule donc :

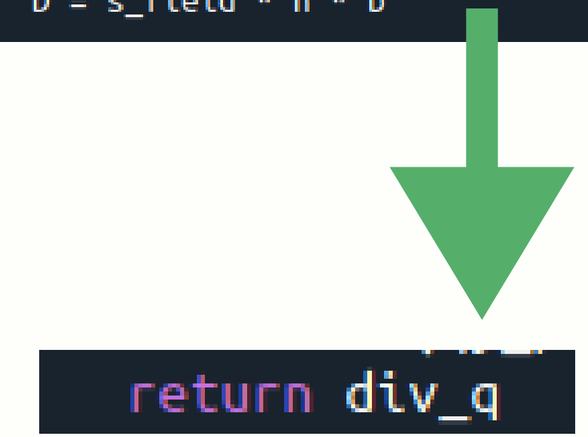
$$D_b(x,t) = \sigma\, n(x,t)\, b(x,t)$$

Le milieu étant hétérogène, on introduit un bruit spatial :

$$\sigma(x) = \sigma_0 \left(1 + \beta\, \xi(x)\right), \quad \xi(x) \in [-1, 1]$$

```python
def nonlinear_diff_b(b, n, s_field, dx, dy):

    D = s_field * n * b
```



```python
    return div_q
```

```python
Delta_static = 2.0 * r * (U1 + U2 - 1.0)
s_field = np.clip(s0 * (1.0 + Delta_static), 0.0, None)

s_field = np.clip(s_field, 0.0, None)    # s >= 0
```

CI et CL

Les conditions initiales sont typiquement :

```
b = bM * np.exp(-(X**2 + Y**2) / 6.25)
```

$$n(x,0) = n_0$$

$$b(x,0) = \begin{cases} b_0 & \|x - x_0\| < r_0 \\ 0 & \text{sinon} \end{cases}$$

On impose des conditions de Neumann (flux nul) sur $\partial\Omega$:

$$\frac{\partial n}{\partial \nu} = 0, \quad D_b \frac{\partial b}{\partial \nu} = 0$$

```
b_pad = np.pad(b, 1, mode='edge')
```

Simulation Numérique

Le flux discret s'écrit :

$$\nabla \cdot (D\nabla b) = \frac{D_{i+\frac{1}{2},j}(b_{i+1,j} - b_{i,j}) - D_{i-\frac{1}{2},j}(b_{i,j} - b_{i-1,j})}{h^2} + \frac{D_{i,j+\frac{1}{2}}(b_{i,j+1} - b_{i,j}) - D_{i,j-\frac{1}{2}}(b_{i,j} - b_{i,j-1})}{h^2}$$

Ce schéma volumique est déterminant pour produire des asymétries et des branches.

```
db_dx = (b_pad[1:-1, 2:] - b_pad[1:-1, :-2]) / (2*dx)
db_dy = (b_pad[2:, 1:-1] - b_pad[:-2, 1:-1]) / (2*dy)

qx = D * db_dx
qy = D * db_dy

qx_pad = np.pad(qx, 1, mode='edge')
qy_pad = np.pad(qy, 1, mode='edge')
```

Simulation Numérique

Schéma temporel (Euler explicite)

$$n^{k+1} = n^k + \Delta t\left((\nabla^2 n)^k - n^k b^k\right)$$

$$b^{k+1} = b^k + \Delta t\left(\nabla \cdot (D\nabla b)^k + n^k b^k\right)$$

```
n_new = n + dt * (lap_n - react)
```

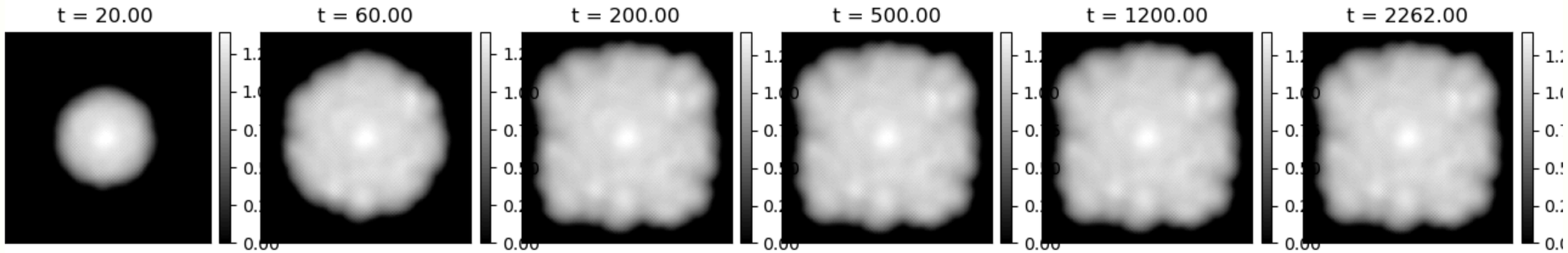
```
diffB = nonlinear_diff_b(b, n, s_field, dx, dy)
b_new = b + dt * (diffB + react)  # + react car c'est le terme de croissance
```

Stabilité

$$\Delta t \leq C \frac{h^2}{\max(D_{i,j})}$$

$$\Delta t \approx 10^{-4}$$

Résultats Obtenus





Conclusion

	Grey-Scott	Kawazaki Murray
Phénomène Modélisé	Chimique (réaction-diffusion)	Dynamique bactérienne réaliste
Dépendance aux paramètres	Du, Dv, F, K	Nutriment, Densité bactérienne
Motifs Obtenus	Bandes, labyrinthes	Motifs dendritiques
Difficulté d'implémentation	Modèle simple	Terme non linéaire, Modèle complexe

Voilà c’est fini !

THANK YOU!

« De la réaction-diffusion aux motifs dendritiques : simulation numérique de la formation des structures bactériennes. »

